Árvores

Fundamentos, Representação e Operações

Prof. Pablo De Chiaro

algoritmos e estrutura de dados 2 (AEDII)

_aula 7

Objetivos da Aula

- Compreender o conceito de árvores e sua importância na estrutura de dados. Estudar a representação e a implementação de árvores binárias. Praticar as operações básicas: inserção, remoção e travessias

POR QUE USAMOS ÁRVORES EM ALGORITMOS?

- **Sistemas de arquivos, JSON, XML/HTML, organizações hierárquicas.**

**recursividade e ordenação se tornam mandatórios.


```
{
    "Account" : "123",
    "data" : [ {
        "Transaction Type" : "Credit",
        "data" : [ {
            "Date" : "2017-01-12",
            "data" : [ {
                "Amount" : 6089.78
            } ]
        }, {
            "Date" : "2017-01-19",
            "data" : [ {
                "Amount" : 482.01
            } ]
        } ]
    }, {
        "Transaction Type" : "Debit",
        "data" : [ {
            "Date" : "2017-01-12",
            "data" : [ {
                "Amount" : 20996.12
            } ]
        }, {
            "Date" : "2017-01-13",
            "data" : [ {
                "Amount" : 17.00
            }, {
                "Amount" : 914.36
            } ]
        }, {
            "Date" : "2017-01-14",
            "data" : [ {
                "Amount" : 671.28
            } ]
        } ]
    } ]
}
```

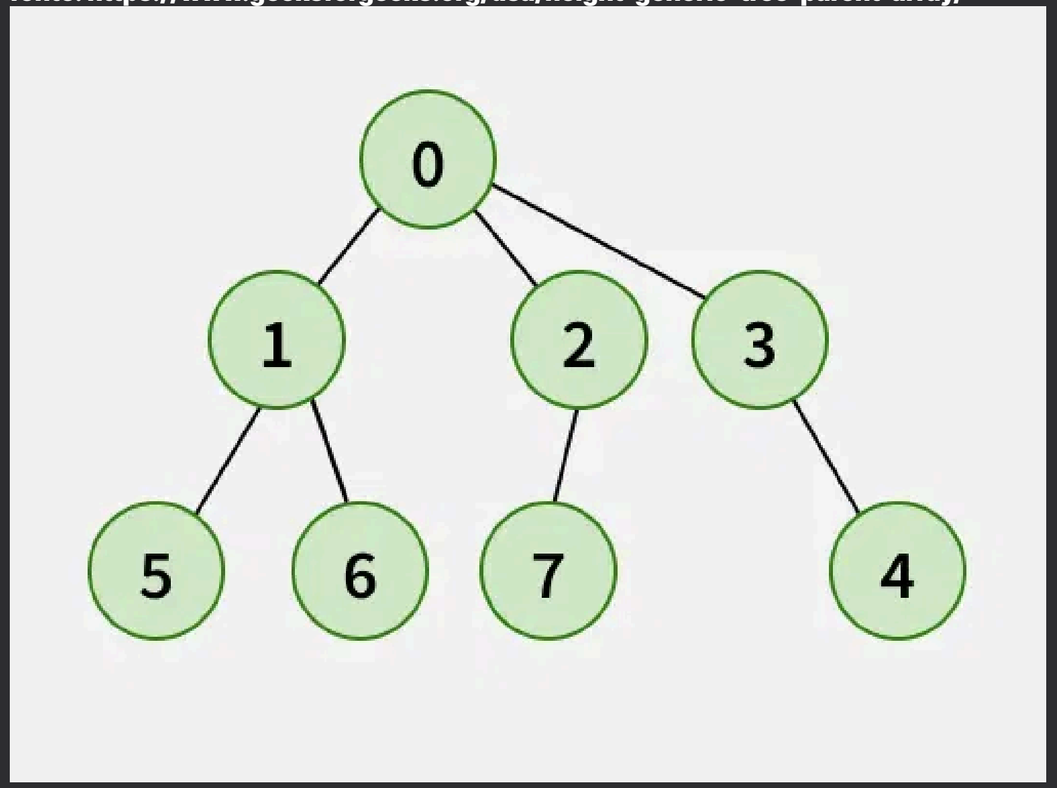
TIPOS BÁSICOS

- Árvore genérica
- Árvore binária
- Árvore binária de busca (BST)

Conceitos: raiz, nó interno, folha, altura, profundidade, grau.

Generic Tree

fonte: https://www.geeksforgeeks.org/dsa/height-generic-tree-parent-array/

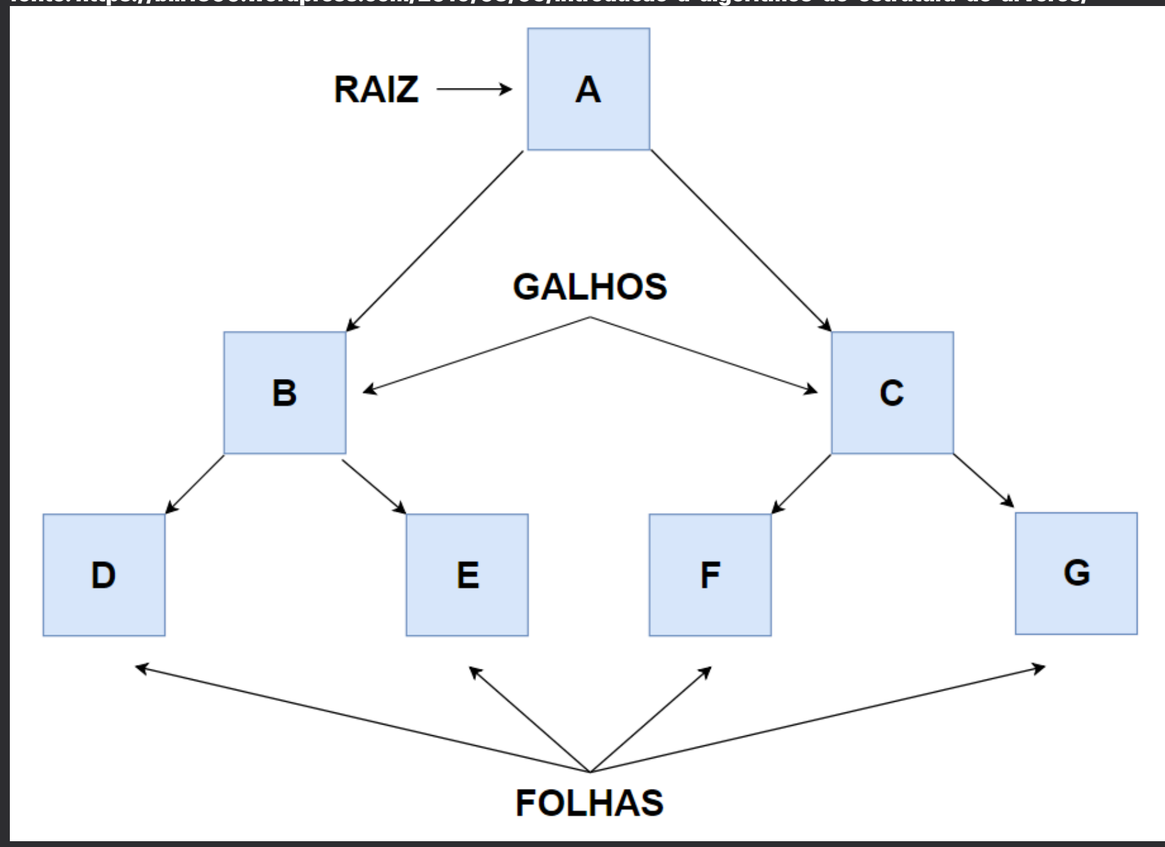


Binárias

- **Conjunto de nós divididos em camadas (raiz, galhos e folhas).**
- **Composição do Nó:** Dado (informação) + 2 ponteiros para filhos.
- **Altura:** Distância da raiz ao descendente mais distante.
- **Profundidade:** Distância da raiz até um nó específico.

Árvore Binária

fonte: https://bill1300.wordpress.com/2019/03/06/introducao-a-algoritmos-de-estrutura-de-arvores/

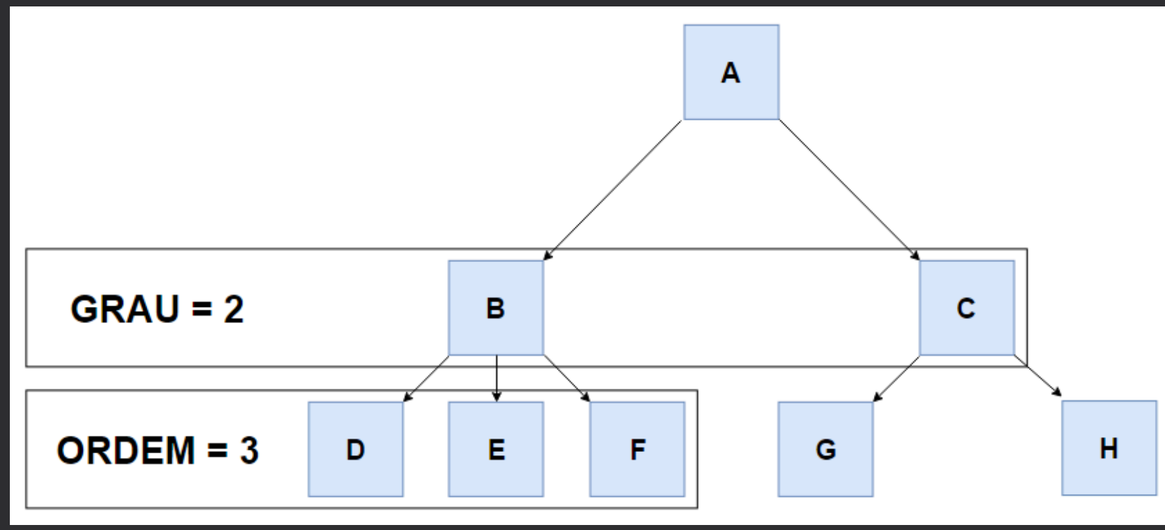


Binárias

- **Características - Grau e Ordem**
- **Grau:** Quantidade de nós que saem diretamente da raiz (ex: Grau 2).
- **Ordem:** O número máximo de filhos que qualquer nó da árvore possui (ex: Ordem 3).

Árvore Binária

fonte: https://bill1300.wordpress.com/2019/03/06/introducao-a-algoritmos-de-estrutura-de-arvores/



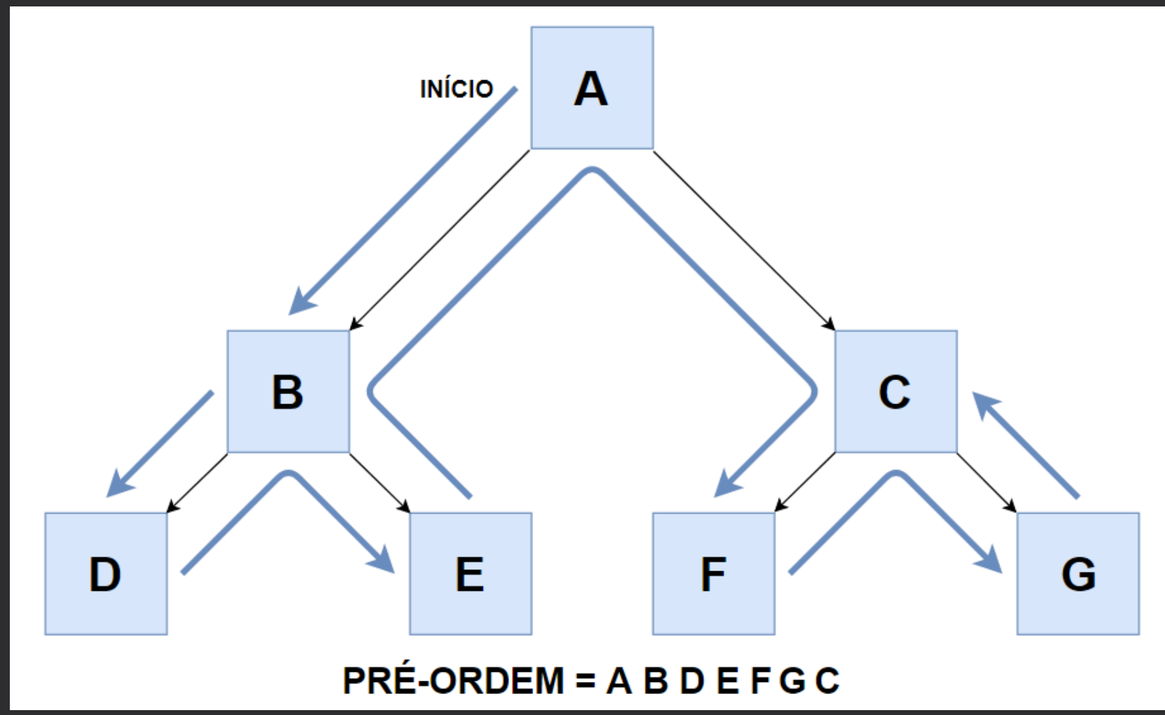
Binárias

- **Percurso em Pré-Ordem**
- **Lógica:** Raiz > Esquerda > Direita.
- **Processo:** Visita a raiz, segue pela esquerda até a folha e retorna para processar a direita.
- **Exemplo da Figura: Sequência**

A-B-D-E-C-F-G.

Árvore Binária

fonte: https://bill1300.wordpress.com/2019/03/06/introducao-a-algoritmos-de-estrutura-de-arvores/



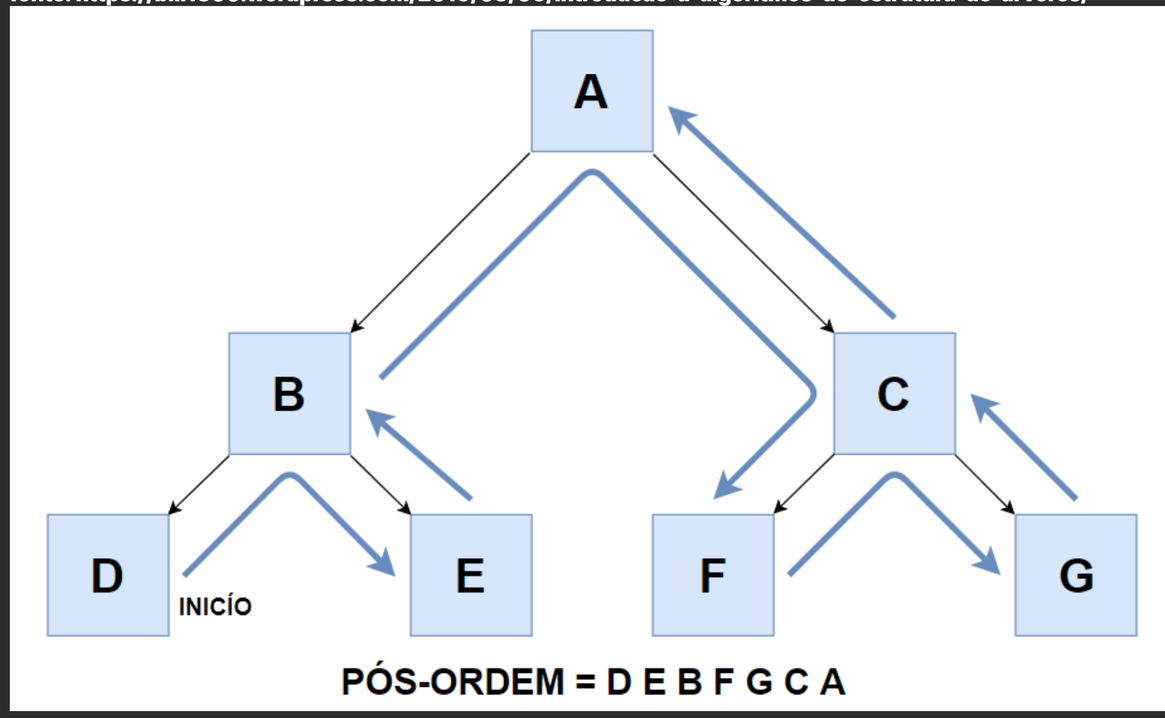
Binárias

- **Percurso em Pós-Ordem**
- **Lógica:** Esquerda > Direita > Raiz.
- **Processo:** Inicia pelo último nó à esquerda, visita o irmão à direita e por fim o nó pai.
- **Exemplo da Figura: Sequência**

D-E-B-F-G-C-A.

Árvore Binária

fonte: https://bill1300.wordpress.com/2019/03/06/introducao-a-algoritmos-de-estrutura-de-arvores/



Árvores N-árias

- Árvores N-árias

Diferença fundamental:

- Binárias: Máximo de 2 filhos; estrutura com chaves e ponteiros (Esq/Dir).
- N-árias: Podem ter N nós filhos a partir de uma raiz.

Binárias

nó (node), estrutura base da árvore

```
class Node:
    def __init__(self, valor):
        self.valor = valor
        self.esq = None
        self.dir = None
```

Binárias

```python
class Node:
    def __init__(self, valor):
        self.valor = valor
        self.esq = None
        self.dir = None
```

```python
def inserir(raiz, valor):
    if raiz is None:
        return Node(valor)
    if valor < raiz.valor:
        raiz.esq = inserir(raiz.esq, valor)
    else:
        raiz.dir = inserir(raiz.dir, valor)
    return raiz
```

Binárias

Exercício: árvore binária de busca (BST) e realizar travessias

- Criar a árvore com os valores: **[10, 5, 2, 7, 15, 12, 20]**
- Implementar travessias pré, in e pós-ordem.
- Imprimir os caminhos percorridos.

*** Vamos discutir o papel da recursão e estrutura de dados nesse processo.*